ITSA 比賽現場筆記

索引（內容在第幾頁）

XAMPP（PHP / MySQL）開賽前 …… 1
Node.js（Express）開賽前 …… 1
2 小時節奏（建議） …… 1

1 Bootstrap 5 RWD 斷點（Breakpoints） …… 2
怎麼用（mobile-first：往大的方向生效） …… 2
CSS media query 對照（自己寫 CSS 時） …… 2
左右 → 上下堆疊（最常考） …… 2
顯示 / 隱藏、對齊、順序 …… 2
常用工具 class …… 2

2 前端 JS 常用片段（重複出現的部分） …… 3
① fetch 包裝：JSON、401 回登入頁 …… 3
② 用 DOM 建立元素（不會被 XSS） …… 3
③ 單頁切換（SPA）：不重新整理 …… 3
④ 表單：按鈕與 Enter 都能送出 …… 3
⑤ 輪詢更新 / 自動登出計時 …… 3
⑥ 淡出後移除 / 保留下拉選擇 …… 3
⑦ 日期過濾 / 排序 / 時間格式 …… 3
⑧ jQuery 匯入元件（2024 決賽 P1） …… 3
⑨ localStorage / sessionStorage …… 3
⑩ SheetJS 讀 Excel …… 3
2.1 Chart.js 從零開始（長條圖 / 圓餅圖） …… 4
步驟 ①+②：HTML 側（引入、畫布） …… 4
長條圖（bar）最小完整例 …… 4
長條圖：多組 dataset（並排 / 堆疊） …… 4
圓餅圖（pie / doughnut）最小完整例 …… 5
更新資料與重畫（最常考） …… 5
Chart.js 常踩的坑 …… 5

3 PHP 後端模板（XAMPP） …… 6
3.1 資料夾與 .htaccess …… 6
檔案結構（2024 P3 單入口版本） …… 6
.htaccess（/login、/admin 簡潔網址） …… 6
3.2 連線與共用工具（每題都會用） …… 6
PDO 連線（MySQL 版，2026 初賽規格） + 查詢小幫手 …… 6
回應、讀 JSON、網址 …… 6
3.3 登入驗證：Session 版（2026 初賽 GamersHub 風格） …… 6
登入 / 登出 / 保護 …… 6
資料隔離（玩家 A 看不到玩家 B 的資料） …… 7
3.4 登入驗證：JWT 版（2024 決賽 P3，10 分鐘自動登出） …… 7
HS256 JWT：自己用 hash_hmac 簽，不需要套件 …… 7
3.5 API 寫法：兩種骨架 …… 7
A · ?a=動作 + 每次回傳完整 state（最省事） …… 7
B · 依 HTTP 方法（REST 風格，2024 P3） …… 8
3.6 SQL 與交易（組隊、搶位這類「不能超賣」的題目） …… 8
交易 + 鎖列：加入組隊（滿員 / 重複加入檢查） …… 8
常用 SQL 片段 + MySQL / SQLite 差異 …… 8
3.7 PHP 常見錯誤 …… 9

4 Express.js 速查（從零開始） …… 10
4.1 建立專案（指令） …… 10
4.2 最小可用伺服器（照順序理解） …… 10
4.3 req / res 一覽 …… 10
4.4 中介層（middleware）與登入 …… 10
概念：(req, res, next) 依序執行 …… 10
Session 登入（2026 初賽 P1 用的做法） …… 11
頁面路由：先檢查再給頁面 …… 11
JWT（HS256）：不用套件的版本 …… 11
4.5 SQLite（Node 內建 node:sqlite：同步，不用 await / callback） …… 11
node:sqlite 語法 …… 11
路由 + 資料庫（CRUD 完整範例） …… 12
4.6 叫號系統（2026 初賽 P1）重點技巧 …… 12
資料庫層面保證規則（不要只靠 if） …… 12
409 規則與驗證用 curl …… 12

5 PHP ↔ Express 對照，以及 Express 常見錯誤 …… 13
Express 常見錯誤 …… 13
兩邊共通的檢查（交卷前） …… 13

XAMPP（PHP / MySQL）開賽前

- Control Panel 按 **Start**：Apache、MySQL 都要變綠色
- 瀏覽 `http://localhost/` 看得到 XAMPP 首頁
- 專案放 `C:\xampp\htdocs\資料夾\`，網址 `http://localhost/資料夾/`
- 匯入 SQL：`http://localhost/phpmyadmin` → 匯入題目的 sql（資料庫 gamer_hub，root、無密碼）
- `.htaccess` 無效時：`C:\xampp\apache\conf\httpd.conf` 確認 `LoadModule rewrite_module` 沒有被 # 註解，htdocs 的 `AllowOverride All`；改完要 **Restart Apache**
- `C:\xampp\php\php.ini`：`extension=pdo_mysql`、`pdo_sqlite`、`mbstring` 前面不能有分號（;）
- PHP 出錯看 `C:\xampp\apache\logs\error.log`

Node.js（Express）開賽前

- `node -v` ≥ 22.13 才有內建 `node:sqlite`；較舊的版本要 `better-sqlite3`（連 node_modules 帶進場）
- 沒有網路 → `npm install` 會失敗。進場前把裝好 express 的 **node_modules** 做成 starter 資料夾
- 啟動：`npm start`（`package.json` 的 scripts.start）→ `http://localhost:3000`
- `EADDRINUSE` = 3000 被占用 → 關掉舊的 node 或換 port
- Bootstrap / jQuery / Chart.js / SheetJS 全部放本機 `lib/`，**不能用 CDN**
- 存檔 UTF-8；PHP 檔存成「無 BOM」

2 小時節奏（建議）

- 0–5 分：全部題目讀完，圈出配分高且熟的項目
- 先做「能拿分又不會卡住」的，一個功能做完立刻截圖
- 每 30 分鐘用瀏覽器完整走一次流程（登入 → 操作 → 登出）
- 最後 10 分鐘：檔名、相對路徑、截圖、自評表、zip，**不再加功能**
- 卡住超過 10 分鐘 → 先跳下一項，回頭再解

1 Bootstrap 5 RWD 斷點（Breakpoints）

名稱	class 前綴	螢幕寬度 ≥	container 最大寬	常見裝置
X-Small	（無前綴）`col-6`	< 576px（0 起）	100%	手機直向
Small	`sm` `col-sm-6`	≥ 576px	540px	手機橫向
Medium	`md` `col-md-6`	≥ 768px	720px	平板
Large	`lg` `col-lg-6`	≥ 992px	960px	小筆電
X-Large	`xl` `col-xl-6`	≥ 1200px	1140px	桌機
XX-Large	`xxl` `col-xxl-6`	≥ 1400px	1320px	大螢幕

怎麼用（mobile-first：往大的方向生效）

- `col-md-6` = 寬度 ≥ 768px 才佔 50%，更窄時預設 100%（上下堆疊）
- 題目寫「≥ 768px 左右並排，< 768px 上下堆疊」→ `col-12 col-md-6`（2024 決賽 P1）
- 題目寫「< 992px 變上下堆疊」→ `col-12 col-lg-4` 與 `col-12 col-lg-8`（2026 初賽 P2）
- 「≥ 某寬度」→ 用該斷點；「< 某寬度」→ 用下一個更小斷點以下（預設就是堆疊）
- 12 欄格線：4+8、6+6、3+9；同一 row 加起來 > 12 會自動換行
- `container`：到各斷點變成固定最大寬；`container-fluid`：永遠 100%；`container-md`：≥768 才固定
- 一定要有 `<meta name="viewport" content="width=device-width, initial-scale=1">`

CSS media query 對照（自己寫 CSS 時）

- 最小寬度：`@media (min-width: 576px)`、`768px`、`992px`、`1200px`、`1400px`
- 最大寬度（Bootstrap 的 down）：`575.98px`、`767.98px`、`991.98px`、`1199.98px`、`1399.98px`
- 規則：max-width = 下一個斷點 − 0.02px，避免 768px 同時命中兩邊
- 題目「小於 768」→ `@media (max-width: 767.98px)`；「大於等於 768」→ `@media (min-width: 768px)`

左右 → 上下堆疊（最常考）

```
<div class="container-fluid">
  <div class="row g-3">
    <div class="col-12 col-lg-4">左：控制面板</div>
    <div class="col-12 col-lg-8">右：大廳 / 行事曆</div>
  </div>
</div>
<!-- 兩欄各 50%：col-12 col-md-6 -->
<!-- 子元素各佔一半：row-cols-1 row-cols-md-2 -->
```

顯示 / 隱藏、對齊、順序

```
<div class="d-none d-md-block">≥768 才顯示</div>
<div class="d-md-none">只在 <768 顯示</div>
<div class="text-center text-md-end">小螢幕置中、md 以上靠右</div>
<div class="order-2 order-md-1">順序（小螢幕/大螢幕不同）</div>
<div class="table-responsive"><table class="table table-striped">…</table></div>
<nav class="navbar navbar-expand-lg">…</nav>
```

常用工具 class

```
間距  m-2 mt-3 mb-4 mx-auto  p-3 py-4 px-2   (0–5)
間隔  g-3（row 內每欄的 gutter）   g-0（無間距）
彈性  d-flex justify-content-between align-items-center gap-2 flex-wrap
寬度  w-100  w-auto  mw-100        文字  text-center text-end fw-bold
顏色  text-bg-info  bg-success  text-warning  text-secondary
元件  btn btn-primary btn-sm w-100   form-control  form-select
      card card-body   badge text-bg-info   progress / progress-bar
      table table-striped   alert alert-danger   nav nav-tabs
深色  <html data-bs-theme="dark">      (Bootstrap 5.3 才有)
```

2　前端 JS 常用片段（重複出現的部分）

① fetch 包裝：JSON、401 回登入頁

```
async function api(url, options = {}) {
  const res = await fetch(url, {
    credentials: 'same-origin',
    headers: {
      'Content-Type': 'application/json',
      'X-Requested-With': 'fetch',  // PHP 判斷用
    },
    ...options,
  });
  if (res.status === 401) {
    location.href = 'login';
    throw new Error('unauthorized');
  }
  return res;
}
const list = await (await api('team001.php?api=faqs')).json();
await api('team001.php?api=faqs', {
  method: 'POST',
  body: JSON.stringify({ question, answer }),
});
```

② 用 DOM 建立元素（不會被 XSS）

```
const el = (tag, text, cls) => {
  const e = document.createElement(tag);
  if (text != null) e.textContent = text;
  if (cls) e.className = cls;
  return e;
};
// 重畫整個列表
box.replaceChildren(...items.map(buildCard));

// 一定要用字串拼 HTML 時，先跳脫：
const esc = s => String(s).replace(
  /[&<>"']/g, c => '&#' + c.charCodeAt(0) + ';');
box.innerHTML = items
  .map(i => `<li>${esc(i.name)}</li>`).join('');
```

③ 單頁切換（SPA）：不重新整理

```
// CSS 要有：  [hidden] { display: none !important; }
function showView(name) {
  document.querySelectorAll('[data-view]')
    .forEach(v => v.hidden = v.dataset.view !== name);
}
// 頁籤（自己控制，不需要 bootstrap.js）
tab1.onclick = e => {
  e.preventDefault();
  boardV.hidden = false;  calV.hidden = true;
  tab1.classList.add('active');
  tab2.classList.remove('active');
};
// 不要用 <a href="另一頁"> 或 location.reload()
// → 瀏覽器重新整理圖示會轉動，題目會扣分
```

④ 表單：按鈕與 Enter 都能送出

```
// 輸入框放在 <form> 裡，按鈕 type="submit"
form.addEventListener('submit', async (e) => {
  e.preventDefault();                // 不要整頁送出
  const text = input.value.trim();
  if (!text) return;
  /* ...送出... */
  input.value = '';
});
// 自動捲到最底（聊天室）
log.scrollTop = log.scrollHeight;
```

⑤ 輪詢更新 / 自動登出計時

```
setInterval(refresh, 2000);   // 題目常寫「5 秒內自動更新」

// token 過期自動回登入頁（exp 單位：秒）
const { exp } = await (await api(
  'team001.php?api=session')).json();
setTimeout(() => location.href = 'login',
  exp * 1000 - Date.now() + 1000);   // +1 秒確保伺服器也過期
```

⑥ 淡出後移除 / 保留下拉選擇

```
/* CSS */  .fade-out { opacity: 0; transition: opacity .5s; }
card.classList.add('fade-out');
setTimeout(() => card.remove(), 500);

// 重建 <select> 時保留目前選的值
const keep = sel.value;
const opt = g =>
  `<option value="${g.id}">${esc(g.name)}</option>`;
sel.innerHTML = '<option value="">-- 請選擇 --</option>'
  + games.map(opt).join('');
sel.value = keep;
```

⑦ 日期過濾 / 排序 / 時間格式

```
// 'YYYY-MM-DD' 字串可以直接比大小
events
  .filter(e => e.date >= from && e.date <= to)
  .sort((a, b) => a.date.localeCompare(b.date));

// 顯示成  2026/7/7 15:10:14
const p2 = n => String(n).padStart(2, '0');
const fmt = ms => {
  const d = new Date(ms);
  return d.getFullYear() + '/' + (d.getMonth() + 1) + '/' +
    d.getDate() + ' ' + p2(d.getHours()) + ':' +
    p2(d.getMinutes()) + ':' + p2(d.getSeconds());
};
// datetime-local 的值： '2026-07-06T23:20'
```

⑧ jQuery 匯入元件（2024 決賽 P1）

```
$.ajax({ url: 'event_filter.html', dataType: 'html' })
  .done(html => $('#host').html(html))  // 會執行元件的 <script>
  .fail(xhr => $('#host').text('遭遇問題: ' + xhr.status +
    ' ' + (xhr.statusText || 'Not Found')));
// 簡寫：$('#host').load('event_filter.html')

// 元件讀引用方的 JSON 資料
// <script id="events-data" type="application/json">
const events = JSON.parse($('#events-data').text());
$('#from, #to').on('change', render);
// 元件檔：無 html/head/body，只有
// <div class="container"> 與 <script>
```

⑨ localStorage / sessionStorage

```
const read = (k, d) => {
  try { return JSON.parse(localStorage.getItem(k)) ?? d; }
  catch { return d; }
};
const write = (k, v) =>
  localStorage.setItem(k, JSON.stringify(v));
// 登入狀態：關掉分頁就消失
sessionStorage.setItem('session', email);
sessionStorage.removeItem('session');
// 純前端題目才用；有後端不要用它存密碼
```

⑩ SheetJS 讀 Excel

```
const wb = XLSX.read(await file.arrayBuffer(),
  { type: 'array' });
const rows = XLSX.utils.sheet_to_json(
  wb.Sheets[wb.SheetNames[0]], { defval: '' });
// rows = [{ 學號, 姓名, 數學, … }]（欄位名稱＝第一列）
// 數字要 Number(row['數學']) 再計算
// 平均 toFixed(2)；及格率 = 及格人數 / 總人數 * 100
```

2.1 Chart.js 從零開始（長條圖 / 圓餅圖）

Chart.js 畫圖固定三步驟：① **引入本機 js** → ② **HTML 放 `<canvas>`（外層要有高度）** → ③ **`new Chart(canvas, 設定)`**。以下兩種圖的完整語法都可以直接背。

步驟 ①+②：HTML 側（引入、畫布）

```
<!-- ① chart.umd.js 放 lib/（本機檔，比賽不能用 CDN） -->
<script src="lib/chart.umd.js"></script>

<!-- ② canvas 外層一定要「有高度」，否則圖不顯示 -->
<div style="position:relative; height:300px">
  <canvas id="c1"></canvas>          <!-- 長條圖用 c1 -->
</div>
<div style="position:relative; height:300px">
  <canvas id="c2"></canvas>          <!-- 圓餅圖用 c2 -->
</div>
```

長條圖（bar）最小完整例

```
new Chart(document.getElementById('c1'), {
  type: 'bar',                          // 'bar' 長條圖
  data: {
    labels: ['數學', '英語', '國文'],   // x 軸刻度名稱
    datasets: [{                        // 一組 dataset = 一排長條
      label: '成績',                    // 顯示在 legend（圖例）
      data: [88, 76, 91],               // 每根的數值
      backgroundColor: 'rgba(54,162,235,.6)',   // 長條填色
      borderColor: 'rgba(54,162,235,1)',        // 長條邊框
      borderWidth: 1,
      borderRadius: 6,                  // 長條頂端圓角
    }],
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,         // 填滿外層 300px（RWD 才正常）
    plugins: {
      title: { display: true, text: '各科成績' },
      legend: { display: true, position: 'top' },   // 'bottom' 'left' 'right'
    },
    scales: {
      y: { beginAtZero: true, max: 100 },  // y 軸 0～100
    },
  },
});
```

長條圖：多組 dataset（並排 / 堆疊）

```
data: {
  labels: ['一組', '二組', '三組'],
  datasets: [
    { label: '本學期', data: [88, 76, 91], backgroundColor: 'rgba(54,162,235,.6)' },
    { label: '上學期', data: [79, 81, 84], backgroundColor: 'rgba(255,159,28,.6)' },
  ],
},
// → 每格並排兩根長條；legend 顯示「本學期／上學期」
// 改成堆疊（同一格疊在一起）：
options: {
  scales: { x: { stacked: true }, y: { stacked: true } },
}
```

圓餅圖（pie / doughnut）最小完整例

```
new Chart(document.getElementById('c2'), {
  type: 'pie',                          // 'pie' 圓餅；'doughnut' 甜甜圈（中間挖洞）
  data: {
    labels: ['Windows', 'macOS', 'Linux'],   // 每一片的名稱（顯示在 legend）
    datasets: [{                        // 圓餅圖只放「一組」dataset
      label: 'OS 佔比',
      data: [62, 30, 8],                // 每片佔比（自動算角度，不用加總 100）
      backgroundColor: ['#36a2eb', '#ff9944', '#4bc0c0'],   // 一片一個顏色，順序對應 labels
      // cutout: '55%',                 // 只給 doughnut 用：中洞大小（寫在 dataset 內，pie 不用寫）
    }],
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      title: { display: true, text: 'OS 佔比' },
      legend: { position: 'bottom' },
      tooltip: {
        callbacks: {
          // 游標移上去的顯示（預設是「Windows: 62」）
          label: (ctx) => ' ' + ctx.label + ' : ' + ctx.parsed + '%',
        },
      },
    },
    // 注意：圓餅圖「不要寫 scales」（沒有座標軸）
  },
});
```

比較	長條圖 bar	圓餅圖 pie
用途	比較「大小 / 數值」	比較「佔比」
datasets	可多組（並排 / 堆疊）	只放一組
scales	要（`y: { min:0, max:100 }`）	**不要寫**
labels	x 軸刻度	每一片的名稱
背景色	一組一個顏色	**一片一個顏色**（陣列，順序對應）
tooltip 數值	游標移上去自帶	`ctx.parsed` 就是該片的數值

更新資料與重畫（最常考）

```
let chart;                          // 記得存住實例

function render(labels, data) {
  if (chart) chart.destroy();     // 重畫前一定 destroy，否則舊圖疊在上面
  chart = new Chart(document.getElementById('c1'), {
    type: 'bar',
    data: { labels, datasets: [{ data }] },
    options: { responsive: true, maintainAspectRatio: false },
  });
}

// 同一張圖只換數字（型別不變）：不用 destroy
chart.data.labels = ['數學', '英語', '國文'];
chart.data.datasets[0].data = [90, 85, 77];
chart.update();                     // 平滑過渡到新數值
// 換圖型（bar → pie）或換 dataset 結構 → 還是 destroy 再 new
```

Chart.js 常踩的坑

- 外層沒有高度（`height:300px`）→ 圖是 0 高度，看不到；`maintainAspectRatio: false` 才會填滿容器
- 重畫前忘了 `chart.destroy()` → 兩張圖疊在一起、資料重複
- 圓餅圖寫了 `scales` 或放了多組 dataset → 不畫 / 報錯
- 忘了引入本機 `lib/chart.umd.js`，或路徑打錯 → 直接 `Chart is not defined`
- 數字是字串（表單拿到的值）→ 先 `Number(x)`，長條圖會不出來
- 圖上常駐顯示數字要另裝 `chartjs-plugin-datalabels`；沒有就改用表格顯示數字
- 其他圖型：折線圖 `type:'line'`（dataset 加 `borderColor: '#36a2eb', tension: 0.3, fill: false`）、雷達圖 `'radar'`、極座標 `'polarArea'`

3 PHP 後端模板（XAMPP）

3.1 資料夾與 .htaccess

檔案結構（2024 P3 單入口版本）

```
htdocs/team001/
  team001.html   前端畫面（登入 / 管理 / 對話 三個 <main data-view>）
  team001.php    後端：資料庫、JWT、API、頁面導向
  team001.js     前端邏輯
  team001.css
  data/          SQLite 檔（.htaccess 擋住直接下載）
  .htaccess
```

.htaccess（/login、/admin 簡潔網址）

```
# 放在網站根目錄 .htaccess
DirectoryIndex team001.php
RewriteEngine On
RewriteRule ^(login|admin)/?$ team001.php [L]

# data/.htaccess（Apache 2.4）
Require all denied
```

REQUEST_URI 會保留原本的 /login，PHP 用網址最後一段判斷要顯示哪一頁。

3.2 連線與共用工具（每題都會用）

PDO 連線（MySQL 版，2026 初賽規格）＋ 查詢小幫手

```php
<?php
$pdo = new PDO('mysql:host=127.0.0.1;dbname=gamer_hub;charset=utf8mb4', 'root', '', [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,        // 數字維持為數字
]);
function q($sql, $params = []) {
    global $pdo;
    $stmt = $pdo->prepare($sql);                  // 永遠用 ? 佔位，不要字串拼接 SQL
    $stmt->execute($params);
    return $stmt;
}
// q('SELECT * FROM t WHERE id=?', [$id])->fetch();      單筆（沒有 → false）
// q('SELECT * FROM t')->fetchAll();                      多筆
// q('SELECT COUNT(*) FROM t')->fetchColumn();            單一值
// q('INSERT ...', [...]); $pdo->lastInsertId();          新 id        ->rowCount() 影響列數
// SQLite 版：new PDO('sqlite:' . __DIR__ . '/data/app.db')    （自己 CREATE TABLE IF NOT EXISTS）
```

回應、讀 JSON、網址

```php
function json_response($data, $status = 200) {
    http_response_code($status);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode($data, JSON_UNESCAPED_UNICODE);        // 中文不轉成 \uXXXX
    exit;
}
function read_json_body() {                               // fetch 的 JSON body（PUT/DELETE 也是）
    $d = json_decode(file_get_contents('php://input'), true);
    return is_array($d) ? $d : [];
}
function base_url() {                                     // 放在子資料夾也正確
    return rtrim(str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME'])), '/') . '/';
}
function redirect($path) { header('Location: ' . base_url() . $path); exit; }
// 防呆：if (trim($x) === '') json_response(['error' => '不可為空'], 400);
```

3.3 登入驗證：Session 版（2026 初賽 GamersHub 風格）

登入 / 登出 / 保護

```php
<?php
session_start();                // 必須在任何輸出之前

// ---- 登入 ----
$u = q('SELECT id, username, password FROM users WHERE username=?', [$name])->fetch();
// 看題目的 sql：明文 → $u['password'] === $pass ；md5 → md5($pass)；hash → password_verify
if (!$u || $u['password'] !== $pass) json_response(['error' => '帳號或密碼錯誤'], 401);
session_regenerate_id(true);
$_SESSION['uid']  = $u['id'];
$_SESSION['name'] = $u['username'];

// ---- 保護（API 和頁面都要）----
$uid = $_SESSION['uid'] ?? null;
if (!$uid) {
    if (($_SERVER['HTTP_X_REQUESTED_WITH'] ?? '') === 'fetch') {
        json_response(['error' => 'unauth'], 401);   // fetch → 401
    }
    redirect('index.html');                          // 直接輸入網址 → 導回登入頁
}

// ---- 登出 ----
$_SESSION = [];
session_destroy();
```

資料隔離（玩家 A 看不到玩家 B 的資料）

```php
// 所有查詢都帶 $uid（從 session 來，絕不從前端傳來）
q('SELECT g.id, g.name FROM user_games ug
     JOIN games g ON g.id = ug.game_id
    WHERE ug.user_id = ?', [$uid])->fetchAll();

// 刪除 / 更新也要帶 user_id，避免改到別人的資料
q('DELETE FROM user_games WHERE user_id=? AND game_id=?', [$uid, (int)$in['id']]);

// 角色檢查（老師 / 學生）
if ($_SESSION['role'] !== 'teacher') json_response(['error' => '沒有權限'], 403);
```

3.4　登入驗證：JWT 版（2024 決賽 P3，10 分鐘自動登出）

HS256 JWT：自己用 hash_hmac 簽，不需要套件

```php
const JWT_SECRET = 'change-this-secret'; const TOKEN_TTL = 600; const COOKIE_NAME = 'token';
function b64url_encode($d) { return rtrim(strtr(base64_encode($d), '+/', '-_'), '='); }
function b64url_decode($d) { return base64_decode(strtr($d, '-_', '+/')); }
function sign_jwt($data) { return b64url_encode(hash_hmac('sha256', $data, JWT_SECRET, true)); }

function create_token($sub, $ttl = TOKEN_TTL) {
    $now = time();
    $h = b64url_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']));
    $p = b64url_encode(json_encode(['sub' => $sub, 'iat' => $now, 'exp' => $now + $ttl]));
    return "$h.$p." . sign_jwt("$h.$p");
}
function verify_token($token) {                // 通過回 payload；錯誤 / 過期回 null
    $parts = explode('.', (string)$token);
    if (count($parts) !== 3) return null;
    [$h, $p, $s] = $parts;
    if (!hash_equals(sign_jwt("$h.$p"), $s)) return null;      // 簽章不符（被竄改）
    $d = json_decode(b64url_decode($p), true);
    return (is_array($d) && ($d['exp'] ?? 0) > time()) ? $d : null;
}
function current_session() { return verify_token($_COOKIE[COOKIE_NAME] ?? null); }
function require_auth() {                                  // 保護 API
    $s = current_session();
    if ($s === null) json_response(['error' => '請先登入'], 401);
    return $s;
}
// 登入成功：把 token 放進 HttpOnly Cookie（前端 JS 讀不到，較安全）
setcookie(COOKIE_NAME, create_token($username), [
    'expires' => time() + TOKEN_TTL, 'path' => '/', 'httponly' => true, 'samesite' => 'Lax',
]);
json_response(['ok' => true]);
// 頁面保護（伺服器端 302）：
if ($page === 'admin' && current_session() === null) redirect('login');
if ($page === 'login' && current_session() !== null) redirect('admin');
```

3.5　API 寫法：兩種骨架

A・?a=動作 ＋ 每次回傳完整 state（最省事）

```php
$a  = $_GET['a'] ?? '';
$in = read_json_body();
switch ($a) {
    case 'add':
        $name = trim($in['name'] ?? '');
        if ($name === '') json_response(['error' => '請輸入'], 400);
        q('INSERT IGNORE INTO games(name) VALUES (?)', [$name]);
        break;                          // 不 exit → 往下回傳最新 state
    case 'remove':
        q('DELETE FROM user_games WHERE user_id=? AND game_id=?',
          [$uid, (int)($in['id'] ?? 0)]);
        break;
    case 'state': break;
    default: json_response(['error' => 'bad action'], 400);
}
json_response([                         // 前端拿到後整包重畫，不用管局部更新
    'me'    => $_SESSION['name'],
    'favs'  => q('SELECT ...', [$uid])->fetchAll(),
    'lobby' => q('SELECT ...', [$uid])->fetchAll(),
]);
```

B · 依 HTTP 方法（REST 風格，2024 P3）

```php
// team001.php?api=faqs        GET 列表 / POST 新增
// team001.php?api=faqs&id=3   PUT 更新 / DELETE 刪除
function api_faqs() {
    require_auth();
    $id = (int)($_GET['id'] ?? 0);
    switch ($_SERVER['REQUEST_METHOD']) {
        case 'GET':
            json_response(faq_list());
        case 'POST':
            [$q, $a] = read_faq_body();              // 空值回 400
            json_response(['id' => faq_add($q, $a)], 201);
        case 'PUT':
            [$q, $a] = read_faq_body();
            if (!faq_update($id, $q, $a)) json_response(['error' => '找不到'], 404);
            json_response(['id' => $id]);
        case 'DELETE':
            if (!faq_delete($id)) json_response(['error' => '找不到'], 404);
            http_response_code(204); exit;           // 204 不能有內容
    }
    json_response(['error' => 'Method Not Allowed'], 405);
}
// 單入口頁面：由網址最後一段決定顯示哪頁
function detect_page() {
    $last = basename(rtrim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/'));
    return in_array($last, ['login', 'admin'], true) ? $last : 'chat';
}
function serve_page($page) {
    $html = file_get_contents(__DIR__ . '/team001.html');
    $html = str_replace('<!-- BASE -->', '<base href="' . htmlspecialchars(base_url()) . '">', $html);
    $html = str_replace('<body data-page="chat">', '<body data-page="' . $page . '">', $html);
    header('Content-Type: text/html; charset=utf-8'); echo $html;
}
```

3.6　SQL 與交易（組隊、搶位這類「不能超賣」的題目）

交易 + 鎖列：加入組隊（滿員 / 重複加入檢查）

```php
$pdo->beginTransaction();
$r = q('SELECT * FROM recruits WHERE id=? FOR UPDATE', [$id])->fetch();   // 鎖住這一列
$c = q('SELECT COUNT(*) FROM recruit_members WHERE recruit_id=?', [$id])->fetchColumn();
$j = q('SELECT 1 FROM recruit_members WHERE recruit_id=? AND user_id=?', [$id, $uid])->fetch();
if (!$r || $c >= $r['max_players'] || $j) {
    $pdo->rollBack();
    json_response(['error' => '無法加入'], 400);                          // 或 409
}
q('INSERT INTO recruit_members(recruit_id, user_id) VALUES (?, ?)', [$id, $uid]);
$pdo->commit();
// 狀態不存欄位，用人數即時算：  cnt >= max → 滿員； cnt >= min → 已成局； 否則招募中
```

常用 SQL 片段 + MySQL / SQLite 差異

```sql
-- 子查詢算人數、是否已加入
(SELECT COUNT(*) FROM recruit_members m WHERE m.recruit_id = r.id) AS cnt,
EXISTS(SELECT 1 FROM recruit_members m
       WHERE m.recruit_id = r.id AND m.user_id = ?) AS joined
-- 別名不能直接放 WHERE → 包一層
SELECT * FROM ( <上面那個查詢> ) t WHERE cnt < mx AND joined = 1 ORDER BY start_time;

-- 新增不重複（欄位要有 UNIQUE）
INSERT IGNORE INTO games(name) VALUES (?)         -- MySQL
INSERT OR IGNORE INTO games(name) VALUES (?)      -- SQLite

-- 建表差異
id INT AUTO_INCREMENT PRIMARY KEY                 -- MySQL
id INTEGER PRIMARY KEY AUTOINCREMENT              -- SQLite
-- 時間： NOW() (MySQL) / datetime('now') (SQLite)
-- datetime-local 的值 '2026-07-06T23:20' → str_replace('T', ' ', $t) 再存
```

3.7 PHP 常見錯誤

症狀	原因與處理
`Cannot modify header information – headers already sent`	`<?php` 前有空白 / BOM，或已經 `echo` 才呼叫 `header()` / `session_start()` / `setcookie()`。檔案存成 UTF-8 **無 BOM**；純 PHP 檔不要寫結尾 `?>`
JSON 壞掉、前端 `JSON.parse` 失敗	PHP 的 Notice / Warning 印在 JSON 前面。看 `error.log`；上線前 `ini_set('display_errors', '0');`
中文變成問號 / 亂碼	連線 `charset=utf8mb4`；`header('Content-Type: …; charset=utf-8')`；HTML `<meta charset="UTF-8">`；資料庫欄位 utf8mb4
PUT / DELETE 拿不到資料	`$_POST` 只收表單。JSON 請用 `file_get_contents('php://input')`；id 放在 `?id=`
`could not find driver`	`php.ini` 啟用 `extension=pdo_mysql`（或 `pdo_sqlite`），**Restart Apache**
`/login` 404、`.htaccess` 沒作用	`mod_rewrite` 沒開 / `AllowOverride None`。備案：直接用 `team001.php?...` 的網址，或在 `httpd.conf` 加規則
MySQL `Access denied` / 連不上	MySQL 沒 Start；帳號 root、密碼空字串；host 用 `127.0.0.1`；port 被占用改 3306
子資料夾部署時 CSS / JS / API 找不到	用相對路徑；頁面用 `<base href="<?= base_url() ?>">`；網址不要多加結尾斜線

4　Express.js 速查（從零開始）

Express ＝ Node.js 的網頁伺服器套件。概念跟 PHP 一樣：**請求進來 → 驗證 → 查資料庫 → 回 JSON**。差別是 PHP 每個網址是一個檔案，Express 是在**一支檔案裡註冊路由**。

4.1　建立專案（指令）

```
mkdir queue && cd queue
npm init -y                      # 產生 package.json
npm install express              # 沒網路：直接複製事先準備好的 node_modules
node server.js                   # 啟動；改了程式要 Ctrl+C 再跑一次
node --watch server.js           # 存檔自動重啟（Node 18+）

// package.json 要有這段，才能用 npm start
"scripts": { "start": "node server.js" },
"dependencies": { "express": "^4.21.2" }

// 目錄
server.js    public/（css、js、前端 html，會直接公開）
views/       （要先檢查登入才給的頁面，不放 public）
```

4.2　最小可用伺服器（照順序理解）

```
const express = require('express');
const path = require('path');
const app = express();

app.use(express.json());                    // ① 解析 JSON body → req.body（一定要）
app.use(express.static(path.join(__dirname, 'public')));   // ② public/ 檔案直接對外

app.get('/api/items', (req, res) => {       // ③ 路由：方法 + 路徑 + 處理函式
  res.json([{ id: 1, name: 'a' }]);         //    200 + JSON
});

app.post('/api/items', (req, res) => {
  const { name } = req.body || {};          // 前端 fetch 的 JSON body
  if (!name) return res.status(400).json({ error: '缺少 name' });   // return 一定要寫
  res.status(201).json({ id: 2, name });
});

app.listen(3000, () => console.log('http://localhost:3000'));      // ④ 最後啟動
```

4.3　req / res 一覽

要做的事	寫法	說明
取 JSON body	`req.body.name`	需要 `app.use(express.json())`；沒有就是 `undefined`
取路徑參數	`req.params.id`	路由寫 `app.put('/api/items/:id', …)`，網址 `/api/items/3` → `'3'`（字串，用 `Number()`）
取查詢字串	`req.query.q`	網址 `?q=abc&page=2`
取 Cookie（原始字串）	`req.headers.cookie`	Express 不會自動解析，要自己用正規表示式取（見 4.4）
回 JSON	`res.json(obj)`	預設 200；要改狀態碼 → `res.status(201).json(obj)`
只回狀態碼	`res.status(204).end()`	204 / 登出 / 刪除成功
回文字 / 檔案	`res.send('ok')`　`res.sendFile(絕對路徑)`	`res.sendFile(path.join(__dirname, 'views', 'admin.html'))`
導向	`res.redirect('/login')`	302。伺服器端導向，瀏覽器不會先看到內容
設定 Cookie	`res.setHeader('Set-Cookie', '…')`	`sid=xxx; HttpOnly; Path=/; SameSite=Lax`；刪除：`Max-Age=0`
路由方法	`app.get / post / put / delete`	不符合的方法 / 路徑 → Express 自動回 404

4.4　中介層（middleware）與登入

概念：(req, res, next) 依序執行

```
// 中介層 = 一個函式；通過就 next()，不通過就直接回應（不呼叫 next）
function auth(req, res, next) {
  if (sessions.has(sidOf(req))) return next();       // 通過 → 交給下一個處理函式
  res.status(401).json({ error: '請先登入' });          // 擋下
}
// 放在路由中間 → 該路由受保護（API 層驗證，不能只在前端隱藏按鈕）
app.get ('/api/admin/history',  auth, (req, res) => { /* ... */ });
app.post('/api/admin/next',     auth, (req, res) => { /* ... */ });

// 所有路由之後放一個 404（可選）
app.use((req, res) => res.status(404).json({ error: 'Not Found' }));
```

Session 登入（2026 初賽 P1 用的做法）

```
const crypto = require('crypto');
const sessions = new Set();                          // 存在記憶體；重啟伺服器就失效
const sidOf = (req) =>
  (/(?:^|;\s*)sid=([^;]+)/.exec(req.headers.cookie || '') || [])[1];

app.post('/api/login', (req, res) => {
  const { username, password } = req.body || {};
  if (username !== 'admin' || password !== 'password123')
    return res.status(401).json({ error: '帳號或密碼錯誤' });
  const sid = crypto.randomBytes(24).toString('hex');
  sessions.add(sid);
  res.setHeader('Set-Cookie', `sid=${sid}; HttpOnly; Path=/; SameSite=Lax`);
  res.json({ ok: true });
});
app.post('/api/logout', auth, (req, res) => {
  sessions.delete(sidOf(req));                       // 伺服器端銷毀 → 舊 Cookie 重放也失效
  res.setHeader('Set-Cookie', 'sid=; Path=/; Max-Age=0');
  res.status(204).end();
});
```

頁面路由：先檢查再給頁面

```
app.get('/', (req, res) =>
  res.sendFile(path.join(__dirname, 'views', 'chat.html')));
app.get('/login', (req, res) => {
  if (sessions.has(sidOf(req))) return res.redirect('/admin');
  res.sendFile(path.join(__dirname, 'views', 'login.html'));
});
app.get('/admin', (req, res) => {
  if (!sessions.has(sidOf(req))) return res.redirect('/login');
  res.sendFile(path.join(__dirname, 'views', 'admin.html'));
});
// 放在 views/ 的檔案不會被 express.static 直接公開
// 放在 public/ 的任何檔案，任何人輸入網址都能下載
```

JWT（HS256）：不用套件的版本

```
const crypto = require('crypto');
const SECRET = 'change-me';
const b64  = (o) => Buffer.from(JSON.stringify(o)).toString('base64url');
const sign = (d) => crypto.createHmac('sha256', SECRET).update(d).digest('base64url');
function createToken(sub, ttl = 600) {
  const now = Math.floor(Date.now() / 1000);
  const h = b64({ alg: 'HS256', typ: 'JWT' }), p = b64({ sub, iat: now, exp: now + ttl });
  return `${h}.${p}.${sign(h + '.' + p)}`;
}
function verifyToken(t) {
  const [h, p, s] = String(t || '').split('.');
  if (!s || sign(h + '.' + p) !== s) return null;          // 簽章不符
  try {
    const d = JSON.parse(Buffer.from(p, 'base64url').toString());
    return d.exp > Date.now() / 1000 ? d : null;           // 過期 → null
  } catch { return null; }
}
// 套件版：npm i jsonwebtoken
//   jwt.sign({ sub: 'admin' }, SECRET, { expiresIn: '10m' })    jwt.verify(token, SECRET) (錯誤會 throw)
// Cookie：`token=${t}; HttpOnly; Path=/; Max-Age=600`
```

4.5　SQLite（Node 內建 node:sqlite：同步，不用 await / callback）

node:sqlite 語法

```
const { DatabaseSync } = require('node:sqlite');          // Node ≥ 22.13
const db = new DatabaseSync(path.join(__dirname, 'app.db'));   // 檔案不存在會自動建立

db.exec(`CREATE TABLE IF NOT EXISTS faqs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  question TEXT NOT NULL, answer TEXT NOT NULL)`);

const one  = db.prepare('SELECT * FROM faqs WHERE id = ?').get(id);     // 單筆；沒有 → undefined
const rows = db.prepare('SELECT * FROM faqs ORDER BY id').all();        // 多筆 → 陣列
const r = db.prepare('INSERT INTO faqs(question, answer) VALUES (?, ?)').run(q, a);
Number(r.lastInsertRowid)    // 新 id
r.changes                    // 影響幾列；0 → 回 404
// 套件版 better-sqlite3：require('better-sqlite3'); new Database(file); 其餘 prepare/get/all/run 完全相同
```

路由 + 資料庫（CRUD 完整範例）

```
app.get('/api/faqs', auth, (req, res) => {
  res.json(db.prepare('SELECT * FROM faqs ORDER BY id').all());
});
app.post('/api/faqs', auth, (req, res) => {
  const { question, answer } = req.body || {};
  if (!question || !answer) return res.status(400).json({ error: '不可為空' });
  const r = db.prepare('INSERT INTO faqs(question, answer) VALUES (?, ?)').run(question, answer);
  res.status(201).json({ id: Number(r.lastInsertRowid), question, answer });
});
app.put('/api/faqs/:id', auth, (req, res) => {
  const { question, answer } = req.body || {};
  const r = db.prepare('UPDATE faqs SET question=?, answer=? WHERE id=?')
              .run(question, answer, Number(req.params.id));
  if (r.changes === 0) return res.status(404).json({ error: '找不到' });
  res.json({ ok: true });
});
app.delete('/api/faqs/:id', auth, (req, res) => {
  const r = db.prepare('DELETE FROM faqs WHERE id=?').run(Number(req.params.id));
  res.status(r.changes ? 204 : 404).end();
});
```

4.6　叫號系統（2026 初賽 P1）重點技巧

資料庫層面保證規則（不要只靠 if）

```
-- 同一時間最多一筆「服務中」：部分唯一索引，違反時 INSERT/UPDATE 直接失敗
CREATE UNIQUE INDEX one_serving ON tickets(status) WHERE status = 'serving';

-- 號碼：各類別獨立、遞增、不重置（票永遠不刪除，所以 MAX 就是上次號碼）
SELECT MAX(seq) FROM tickets WHERE prefix = ?     →  seq = (MAX ?? 0) + 1
number = prefix + String(seq).padStart(3, '0')    // A001；超過 999 自然變 A1000

-- 先進先出（所有類別共用一個佇列）
SELECT * FROM tickets WHERE status='waiting' ORDER BY created_at, id LIMIT 1;

-- 狀態只能往前：waiting → serving → completed
UPDATE tickets SET status='serving', called_at=? WHERE id=?
```

409 規則與驗證用 curl

```
// 下一位：已有 serving → 409；沒有 waiting → 409（訊息不同）
// 完成服務：沒有 serving → 409
// 取號：service 不在白名單 → 400，而且不能產生任何號碼

curl -i -X POST localhost:3000/api/tickets -H "Content-Type: application/json" -d "{\"service\":\"諮詢\"}"
curl -i -c jar.txt -X POST localhost:3000/api/login -H "Content-Type: application/json" -d "
{\"username\":\"admin\",\"password\":\"password123\"}"
curl -i -b jar.txt -X POST localhost:3000/api/admin/next
curl -i -b jar.txt -X POST localhost:3000/api/logout
curl -i -b jar.txt localhost:3000/api/admin/history      # 登出後重放舊 Cookie → 必須 401
// 評審會用 curl 直接打 API：只在前端把按鈕 disabled 是不算分的
// 時間存 Date.now()（毫秒），輸出才格式化成 YYYY/M/D HH:mm:ss
// 推測（決賽可能的變化）：多櫃台 → 加 counter_id，索引改成 ON tickets(counter_id) WHERE status='serving'
```

5　PHP ↔ Express 對照，以及 Express 常見錯誤

功能	PHP	Express
啟動方式	放進 `htdocs`，Apache 提供	`node server.js` / `npm start`，自己 `app.listen(3000)`
一個網址對應什麼	一個 `.php` 檔（或 `.htaccess` 改寫）	`app.get('/路徑', handler)`
判斷 HTTP 方法	`$_SERVER['REQUEST_METHOD']`	`app.get / post / put / delete`
JSON body	`json_decode(file_get_contents('php://input'), true)`	`req.body`（先 `app.use(express.json())`）
查詢字串 / 路徑參數	`$_GET['id']`	`req.query.id` / `req.params.id`
回 JSON 與狀態碼	`json_response($d, 201)`	`res.status(201).json(d)`
提早結束	`exit;`	`return res…`（回應後一定要 `return`）
導向	`header('Location: /login'); exit;`	`res.redirect('/login')`
登入狀態	`$_SESSION['uid']`（PHP 自動管理）	自己存：`Set` / `Map` / JWT Cookie
讀 / 寫 Cookie	`$_COOKIE['token']` / `setcookie()`	`req.headers.cookie`（自己 parse） / `res.setHeader('Set-Cookie', …)`
受保護的 API	`require_auth();`（函式開頭呼叫）	`app.get(path, auth, handler)`（中介層）
資料庫	PDO：`prepare → execute → fetch`	`db.prepare(sql).get / all / run`
目前檔案位置	`__DIR__`	`__dirname`
字串內插	`"$name"`	`` `${name}` `` （反引號）

Express 常見錯誤

- `req.body` 是 `undefined` → 漏了 `app.use(express.json())`，或前端沒送 `Content-Type: application/json`
- `Cannot set headers after they are sent` → 同一個請求回應了兩次，通常是 `if (…) res.status(400)…` **漏了** `return`
- `EADDRINUSE` → port 被占用，關掉舊的 node 視窗，或換 port
- 改了程式沒有變化 → 沒重啟（用 `node --watch`）；瀏覽器快取 → Ctrl+F5
- `res.sendFile` 路徑錯誤 → 必須是**絕對路徑**：`path.join(__dirname, …)`
- 路由順序：`/api/items/:id` 會吃掉 `/api/items/new`；固定路徑要寫在參數路徑前面
- `req.params.id` 是字串，比較或查詢前用 `Number()`
- Express 5 的萬用字元要命名：`app.get('/*splat', …)`（Express 4 可寫 `'*'`）。想避開差異 → `npm i express@4`

兩邊共通的檢查（交卷前）

- 未登入直接打 API → 401 / 打頁面 → 302 回登入頁
- 登出後，舊 Cookie / token 不能再使用
- 錯誤情境都測：空值、重複送出、不存在的 id、不在白名單的值
- 題目指定文字（alert、placeholder、預設資料）逐字比對
- 用 F12 → Network 看每個請求的狀態碼與回應內容
- 開兩個瀏覽器（或無痕）測「資料隔離」與「即時更新」
- 重新整理後資料還在（真的寫進資料庫了）
- 關掉再重啟伺服器，畫面還能還原（叫號系統 I6）